

PPS-2(LAB-11)[PART-2]

QUE-1: Validate Color.

Solu: `#include<iostream>`

`#include<string>`

`#include<exception>`

`#include<cstring>`

`using std::cout;`

`using std::cin;`

`using std::string;`

`using std::endl;`

`class InvalidColor: public std::exception {`

`public:`

`virtual const char* what() const throw()`

`{`

`return "InvalidColor";`

`}`

`};`

```

void fill_color(string color) {
    string valid_colors[3] = {"red", "green", "blue"};
    bool is_valid_color = false;
    for(int i = 0; i < 3; i++)
        if(valid_colors[i] == color) {
            is_valid_color = true;
            break;
        }
    if(!is_valid_color) {
        throw *(new InvalidColor);
    }
    /* logically code to fill color should be here
       you dont need to write any code here */
}

```

```

int main() {

    string color;
    cin >> color;
    try{
        fill_color(color);
    }
    catch(InvalidColor& e) {
        cout << e.what() << endl;
    }
}

```

```
}

return 0;

}
```

QUE-2: Throwing exception of user defined type.

Solu: `#include<iostream>`

`#include<string>`

`#include<exception>`

`#include<cstdlib>`

`using std::cout;`

`using std::cin;`

`using std::string;`

`using std::endl;`

`using namespace std;`

`/* Write your code here */`

`class MemoryAllocationFailed : public exception {`

`virtual const char* what() const throw() {`

`return "MemoryAllocationFailed";`

`}`

```
};
```

```
void* allocate_memory(long size) {  
    void* ptr = malloc(size);  
    if(ptr == nullptr) {  
        throw MemoryAllocationFailed();  
    }
```

```
    return ptr;
```

```
}
```

```
int main() {
```

```
    void *vptr;
```

```
    try {
```

```
        vptr = allocate_memory(100);
```

```
        free(vptr);
```

```
        vptr = allocate_memory(1000000000000000);
```

```
        free(vptr);
```

```
    }
```

```
    catch(std::exception &e) {
```

```
        cout << e.what();
```

```
}
```

```
return 0;
```

```
}
```

PPS-2(LAB-12)

QUE-1: Sort and print function templates I.

Solu: `#include<iostream>`

`#include<string>`

`using std::cout;`

`using std::cin;`

`using std::string;`

`using std::endl;`

`template <typename T>`

`void sort(T arr[], int size) {`

`for (int i = 0; i < size - 1; i++) {`

`for (int j = 0; j < size - i - 1; j++) {`

`if (arr[j] > arr[j + 1]) {`

`T temp = arr[j];`

`arr[j] = arr[j + 1];`

`arr[j + 1] = temp;`

`}`

`}`

`}`

`}`

```
template <typename T>
void print(T arr[], int size) {
    for (int i = 0; i < size; i++) {
        cout << arr[i] << " ";
    }
    cout << endl;
}
```

```
int main() {
```

```
    int int_arr[10] = {9, 8, 1, 2, 6, 5, 4, 3, 0, 7};
    sort(int_arr, 10);
    print(int_arr, 10);
```

```
    float float_arr[6] = {9.1, 8.2, 8.3, 7.7, 9.9, 9.1};
    sort(float_arr, 6);
    print(float_arr, 6);
```

```
    string str_arr[3] = {string("XYZ"), string("ABC"), string("PQR")};
    sort(str_arr, 3);
    print(str_arr, 3);
```

```
    return 0;
}
```

QUE-2: Sort and print function templates II.

Solu: #include<iostream>

#include<string>

using std::cout;

using std::cin;

using std::string;

using std::endl;

/ Create function templates for sort and print */*

template <typename T>

void sort(T arr[], int size) {

for (int i = 0; i < size - 1; i++) {

for (int j = 0; j < size - i - 1; j++) {

if (arr[j] > arr[j + 1]) {

T temp = arr[j];

arr[j] = arr[j + 1];

arr[j + 1] = temp;

}

}

}

}


```
template <typename T>
void print(T arr[], int size) {
    for (int i = 0; i < size; i++) {
        cout << arr[i] << " ";
    }
    cout << endl;
}
```

```
template <typename T>
void sort_and_print(T arr[], int size) {
    sort(arr, size);
    print(arr, size);
}
```

```
int main() {
```

```
    int int_arr[10] = {9, 8, 1, 2, 6, 5, 4, 3, 0, 7};
    sort_and_print(int_arr, 10);
```

```
    float float_arr[6] = {9.1, 8.2, 8.3, 7.7, 9.9, 9.1};
    sort_and_print(float_arr, 6);
```

```
    string str_arr[3] = {string("XYZ"), string("ABC"), string("PQR")};
    sort_and_print(str_arr, 3);
```

```
return 0;
```

```
}
```